A Small First Tier for Phone Agents: A Pre-Registered Offline Study of Routing Step-Instructed Actions to a 149M Local Model

Chaitanya Talapalli *chayt128@gmail.com*
Independent researcher

Abstract

Phone agents usually send every step to a frontier language model, although many steps are easy. In an offline, step-instructed setting, we test whether a 149M-parameter model that reads the screen's accessibility tree can answer the steps it is confident about and pass the rest to a frontier model without losing accuracy. All hypotheses, the routing threshold and the evaluation sample were fixed in a dated log before any frontier model saw the test steps. On 1,000 held-out AndroidControl steps, evaluated offline with a human-written instruction for each step, the small model answers 70.7% of steps. Deferring the rest to Claude Opus 5.5, which is shown 16 similar solved training examples, scores 892 against 882 for Opus alone (paired difference +1.0, 95% CI $[-0.2, +2.2]$) at 29.5% of its cost. Deferring to Claude Sonnet 5.5 instead scores 881 against 857 for Sonnet alone and stays within the registered 2-point margin of Opus alone at 14.8% of Opus's cost. A second run of both frontier models replicates these results; they also hold on apps absent from training and, for Opus, on corrected labels. Deferring to the weaker Haiku 5.5 adds nothing over the small model alone. The small model alone scores 869, between few-shot Sonnet and few-shot Opus. The results are limited to single steps: a related cascade lost to a frontier agent on live multi-step tasks, and on a phone CPU the model needs a median 3.4 s per step. We release the model, per-step outputs of every system, and the registration log.

1 Introduction

A phone agent reads the screen, chooses an action, acts, and repeats. The most accurate agents call a frontier model at every step, which costs money, adds a network round trip and sends the screen's contents off the device. Many steps are nevertheless easy: tap the tab labelled "Timers", press "5", go back. If a small local model answered those steps and deferred only the hard ones, most frontier calls would disappear.

Deferring uncertain inputs to a stronger decision maker is well studied in selective classification (Chow, 1970; Geifman & El-Yaniv, 2017) and learning to defer (Mozannar & Sontag, 2020), and routing between cheap and expensive language models is an active area (Chen et al., 2023; Aggarwal et al., 2024; Ding et al., 2024; Ong et al., 2024). Phone agents have recently adopted the idea: OpenPhone (Jiang & Huang, 2025), MAI-UI (Zhou et al., 2025) and CORE (Fan et al., 2025) combine a billion-parameter local model with a cloud model and report end-to-end gains. Two questions remain open. How much of a phone agent's work can a model one to two orders of magnitude smaller take on, step by step, without costing the frontier model accuracy? And is the comparison fair? Small fine-tuned models are routinely compared with frontier models prompted zero-shot (Christianos et al., 2025), a known source of inflated gains (Melis et al., 2018; Dacrema et al., 2019).

We answer both questions on AndroidControl (Li et al., 2024) under a pre-registered protocol (Fig. 1). Our contributions are:

- A routing study on 1,000 held-out steps with three frontier models, replicated with a second run, broken down by unseen apps and action types, and re-scored on corrected labels (Sec. 4).

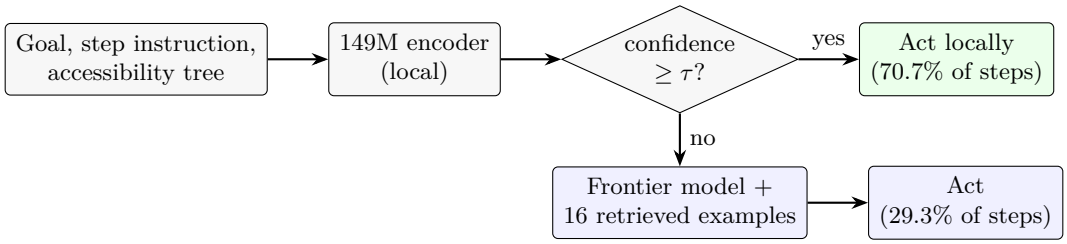


Figure 1: Local-first routing. The small model predicts an action and a confidence; steps at or above the threshold τ are answered locally, the rest by a frontier model shown similar solved examples. The threshold was fixed before the evaluation.

- A fair-baseline study showing that the small model's 6–8 point lead over zero-shot frontier models disappears once those models see similar solved examples (Sec. 5).
- On-device measurements on a phone CPU, a quantized model evaluated end to end, a live phone demonstration, and a live multi-step result that bounds the claims (Secs. 6 and 7).
- The released model, per-step outputs of every system and the registration log (Appendix B).

Neither the model design, which follows candidate selection in MindAct (Deng et al., 2023) and LiMAC (Christianos et al., 2025), nor the deferral rule is new; the contribution is the measurement and the released first-tier model.

2 Related Work

Phone and GUI agents. AndroidControl (Li et al., 2024) provides step-level supervision with out-of-distribution splits; AndroidControl-Curated (Leung et al., 2025) corrects its labels. Screenshot agents such as OS-Atlas (Wu et al., 2024), UI-TARS (Qin et al., 2025), InfiGUI-R1 (Liu et al., 2025) and Ferret-UI Lite (Yang et al., 2025) lead the benchmark. LiMAC (Christianos et al., 2025) is the closest small system. AndroidWorld (Rawles et al., 2025) evaluates agents live. AutoDroid-V2 (Wen et al., 2024) lets a small on-device model write multi-step scripts instead of choosing one action at a time.

Deferral and routing. Selective classification abstains at low confidence (Chow, 1970; Geifman & El-Yaniv, 2017), and learning to defer trains a model to hand cases to an expert (Mozannar & Sontag, 2020). For language models, FrugalGPT (Chen et al., 2023) cascades APIs from cheap to expensive, AutoMix (Aggarwal et al., 2024) escalates after self-verification, and Hybrid LLM (Ding et al., 2024) and RouteLLM (Ong et al., 2024) learn routers. For phone agents, OpenPhone (Jiang & Huang, 2025) escalates hard subtasks from a 3B on-device model to the cloud, MAI-UI (Zhou et al., 2025) hands off when the local trajectory drifts from the user's intent and reports over 40% fewer cloud calls, and CORE (Fan et al., 2025) lets a local model rank screen regions so that the cloud model sees less of the screen. Our local model is far smaller, and we isolate its effect at the step level against a frontier baseline matched for in-domain data.

Fair baselines. Retrieved demonstrations raise frontier models' mobile-agent accuracy substantially (Liu et al., 2026; Xu et al., 2025); we use retrieval only to strengthen the baseline. Under-tuned baselines have inflated reported gains across machine learning (Melis et al., 2018; Dacrema et al., 2019; Musgrave et al., 2020; Lipton & Steinhardt, 2019), and model accuracy depends on prompt format (Sclar et al., 2024), element order (Chi et al., 2024) and reasoning budget (Gema et al., 2025). Computer-use agents vary across reruns (Gonzalez-Pumariega et al., 2026; Khanal et al., 2026), which motivates our replication.

3 Method

Task. Each AndroidControl step provides the task goal, a human-written instruction for the step and the accessibility tree of the current screen. The system outputs one of 11 actions (tap, long press, type text,

scroll in one of four directions, back, home, open app, wait) with its argument. Evaluation is offline: every step starts from the recorded history.

Small model. Each accessibility tree is flattened into at most 220 elements, each rendered as class, visible text or description, hint, resource-id words, a coarse screen position and state flags. A ModernBERT-base encoder (Warner et al., 2024) (149M parameters) reads the instruction, a one-line history (previous action and whether the screen changed) and the goal, followed by the elements, each preceded by a marker token. Three heads predict the action type, the target element (one logit per marker) and, for typed text and app names, a span copied from the instruction. We release the model as Milo Pointer. Training details are in Appendix A. We chose extraction over generation after a registered comparison at equal training data: on steps that fit the generators' context, the encoder with copy heads was 2.4–6.8 points more accurate than LoRA-tuned Qwen2.5-3B and 7B models that generate the action as text, and three training seeds gave $80.8 \pm 0.7\%$ on that subset.

Routing. Confidence is the probability of the predicted action type, multiplied for taps by the probability of the chosen element. A step is answered locally if its confidence is at least $\tau = 0.9854$, the 30th percentile on a separate sample of 1,000 test steps; τ was fixed before any frontier model saw the evaluation steps.

Frontier baseline. Claude Opus 5.5, Sonnet 5.5 and Haiku 5.5 (Anthropic; Claude Fable 5.1, a further Anthropic model, appears in Sec. 5) receive a system prompt describing the actions, the same inputs as the small model, and 16 solved training steps retrieved by TF-IDF similarity of goal and instruction, each with its correct action and, for taps, the target element in words. The routing arms use adaptive reasoning at medium effort through the Batch API; the earlier arms in Table 5 list their effort. We call this setting few-shot (FS).

Metric, statistics and costs. A step is correct if the action type matches the record and, for taps, the centre of the chosen element lies inside the recorded target; typed text must match exactly after normalisation. We report paired differences with Wald 95% intervals and exact McNemar tests; intervals computed by resampling whole episodes are essentially identical. The registered non-inferiority margin is 2 points, about a quarter of the gain Opus gets from examples (7.9 points); because a per-step loss compounds over a task, it suits single-step claims only. Costs are billed tokens at October 2026 Batch API prices (per million input/output tokens before the 50% batch discount: Opus \$4/\$20, Sonnet \$2/\$10, Haiku \$0.10/\$0.50); real-time prices double every figure without changing ratios. On-device compute is not priced.

Pre-registration. Each comparison was written to a dated log, with hypotheses, sample, threshold and margin, and committed to version control before any frontier request on its data. The log is self-hosted rather than held by a third-party registry. As an external check, every frontier run was submitted through the Message Batches API, which records the batch creation time on the provider's servers; for every routing study the registration commit precedes its batch (released table of commits and batch identifiers).

4 Local-First Routing

The evaluation set is 1,000 test steps (579 episodes) drawn at random from the 836 test episodes not used to set τ. We registered that routing to Opus is non-inferior to Opus alone; that at least 60% of steps stay local; and that routing to Sonnet is non-inferior both to Sonnet alone and to Opus alone. Routing to Haiku was registered later, after an internal review.

Results. The small model answers 70.7% of steps (Table 1). Routing to Opus scores 892 against 882 for Opus alone: non-inferior, not significantly better. Routing to Sonnet lifts Sonnet by 2.4 points and reaches Opus alone. A second run of each frontier model replicates every registered result (Table 2); each model's two runs differ in correctness on 21 of 1,000 steps. Routing to Haiku lifts Haiku from 805 to 865 but is no better than the small model alone: a fallback helps only if it is stronger than the small model on the deferred steps, where Opus is right 75.4% of the time and the small model 67.6%. Alone, the small model is 1.3 points behind Opus and outside the non-inferiority bound (-3.4), so the frontier tier is needed to match

Table 1: Correct steps out of 1,000 held-out steps. Run 2 repeats every frontier request unchanged. Cost per 1,000 steps from billed tokens (Batch API).

System	Run 1	Run 2	Answered locally	Cost ($)
Small model alone	869	869	100%	0
Opus 5.5 FS alone	882	879	0%	9.54
Small model → Opus 5.5 FS	**892**	**890**	70.7%	2.82
Sonnet 5.5 FS alone	857	862	0%	4.80
Small model → Sonnet 5.5 FS	**881**	**889**	70.7%	1.41
Haiku 5.5 FS alone	805	–	0%	0.26
Small model → Haiku 5.5 FS	865	–	70.7%	≈0.08

Table 2: Paired differences in points, with 95% CI and exact McNemar p. Last column: registered hypothesis and outcome.

Comparison	Run 1	Run 2	Registered
Small→Opus vs Opus alone	$+1.0$ $[-0.2, +2.2]$, p=0.14	$+1.1$ $[-0.2, +2.4]$	non-inferior: held
Small→Sonnet vs Sonnet alone	$+2.4$ $[+1.1, +3.7]$, p=0.0005	$+2.7$ $[+1.3, +4.1]$	non-inferior: held
Small→Sonnet vs Opus alone	-0.1 $[-1.7, +1.5]$	$+1.0$ $[-0.7, +2.7]$	non-inferior: held
Small→Haiku vs Opus alone	-1.7 $[-3.5, +0.1]$	–	non-inferior: failed
Small→Opus vs small alone	$+2.3$, p=0.011		–
Small alone vs Opus alone	-1.3 $[-3.4, +0.8]$		–

Opus. Routing to Opus costs $2.82 per 1,000 steps (29.5% of Opus alone); deferred steps produce more output tokens (56 vs 31), but input tokens dominate cost.

What stays local. Table 3 breaks run 1 down by action. The small model keeps nearly all scrolls down, waits, back presses, typing and app launches, and 59% of taps. On the steps it keeps, its advantage over Opus comes from the dataset's conventions rather than from taps (Table 4): waiting when the previous action has not yet taken effect (50 vs 46), naming scroll direction by where new content appears (98 vs 92) and going back (33 vs 31), against 337 vs 334 on taps. Opus is slightly better on the kept typing and app-launch steps. The deferred steps are mostly taps, often on icons without text, which a text-only model cannot read.

Unseen apps, corrected labels and other scores. These analyses were not registered. On the 457 steps from apps absent from training, routing to Opus scores 408 against 405 for Opus alone, and routing to Sonnet 403 against 391 for Sonnet alone. On AndroidControl-Curated labels (Leung et al., 2025) (995 matched steps), routing to Opus scores 928 against 925 for Opus alone ($+0.3$, $[-0.8, +1.4]$); routing to Sonnet scores 914, -1.1 ($[-2.5, +0.3]$) against Opus alone, narrowly outside the margin. Over all 8,377 matched test steps the small model scores 92.0% on curated labels and 88.6% on the original ones. Ordering steps by the margin between the top two probabilities or by entropy gives the same accuracy as the registered score (890 and 889 with Opus at 70% local); the action-type probability alone is clearly worse (862).

A learned deferral rule (exploratory). The registered score ignores how sure the model is about copied text, whether the chosen element is an icon without text, and the kind of action. We fit two logistic regressions on the separate sample used to set τ, one predicting whether the small model is right and one whether few-shot Opus is right, from the action-type probability, margin and entropy, the element probability and margin, the copy-span probability, an unlabelled-icon flag, the predicted action type and the screen size. Keeping the 70.7% of steps with the largest predicted advantage of the small model, and testing on the held-out steps, routing to Opus scores 902 (run 1) and 900 (run 2) against 882 and 879 for Opus alone: $+2.0$ ($[+0.6, +3.4]$, $p = 0.006$) and $+2.1$ ($[+0.8, +3.4]$, $p = 0.003$), significantly better rather than only non-inferior. Routing to Sonnet scores 891 and 898. Against the registered rule the learned rule gains 0.9–1.0 points in all four comparisons, each not significant on its own ($p = 0.10$–0.14). Type entropy, the element margin and

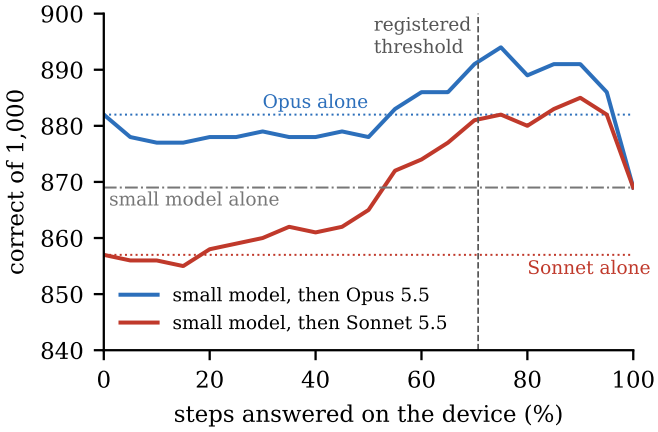


Figure 2: Accuracy of the routed system as the share of locally answered steps varies (run 1). Dotted lines: frontier models alone. The gain appears once the small model keeps more than about half of the steps.

Table 3: Run 1 by recorded action: share answered locally and correct answers on kept and deferred steps.

Action	Steps	Kept	Small model, kept	Opus, kept	Opus, deferred
Tap	605	59%	337/358	334/358	194/247
Scroll down	102	96%	98/98	92/98	4/4
Type text	88	93%	79/82	80/82	1/6
Open app	79	99%	71/78	76/78	0/1
Wait	62	81%	50/50	46/50	10/12
Back	40	82%	33/33	31/33	6/7
Other scrolls	24	33%	3/8	2/8	6/16

screen size carry most weight; icons and wait steps push towards deferral. This rule was not registered and needs a confirmatory test on new data.

5 Fair Baselines: The Small Model Alone

Without examples, Opus 5.5 and Sonnet 5.5 trail the small model by 7.8 and 7.4 points (both $p < 10^{-5}$; Table 5, Fig. 3). Many of their errors are dataset conventions rather than reasoning: 28 “scroll down” instructions answered with scroll up and 27 wait steps answered with a tap. At equal reasoning effort, 16 retrieved examples raise Opus from 814 to 893 (+7.9 pts, $p < 10^{-14}$), including on apps absent from training (320 to 359 of 401). The small model is then 1.5 points behind Opus ($p = 0.17$, CI $[-3.5, +0.5]$) and 0.8 ahead of Fable 5.1 ($p = 0.52$); the registered criterion, that it exceed every few-shot arm, is not met. For reference, on the full test set (8,444 steps) the small model scores 89.8% in-domain and 86.7–87.9% on the unseen-app, -task and -category splits, against 76.8–83.2% for the best fine-tuned PaLM 2S (Li et al., 2024); on a stricter holdout of 72 apps never seen in training it drops from 86.2% to 82.6%.

6 Running on a Phone

We exported the model to ONNX and ran it with ONNX Runtime 1.20 on the CPU of a Pixel 6a (4 threads) over 40 real inputs (median 1,228 tokens). At full precision the outputs match PyTorch on all 40 steps, with a median latency of 4.5 s (90th percentile 11.9 s) and 1.94 GB peak memory. With 8-bit weights (150 MB on disk) latency falls to 3.4 s (11.2 s) and memory to 1.45 GB. Over all 1,000 held-out steps the 8-bit model scores 851 alone (869 at full precision), keeps 66.6% of steps under the unchanged threshold, and routing still holds: 886 against 882 for Opus alone (+0.4, $[-0.7, +1.5]$). On a laptop GPU (Apple M2, fp16) the median

Table 4: Examples from the held-out steps (instructions verbatim; confidence in parentheses).

Instruction and history	Recorded action	Small model	Opus FS
"Click on the convert button"; same instruction as the previous step, screen half unchanged	wait	wait, kept (0.997)	tap
"Scroll up on the screen until you find the ratings & reviews section"	scroll down	scroll down, kept (0.996)	wait
"Click on the headphone icon to listen to news"	tap the icon	deferred (0.635)	tap the icon

Table 5: The separate sample of 1,000 test steps used to set τ. ZS: no examples. Unseen apps: 401 steps.

System	Reasoning effort	Correct	Unseen apps	Cost ($/1k)
Haiku 5.5 FS	low	797	–	0.25
Opus 5.5 ZS	low	800	320	5.93
Sonnet 5.5 ZS	low	804	–	3.00
Opus 5.5 ZS	medium	814	–	6.08
Fable 5.1 FS	medium	870	350	23.36
Small model	–	878	350	0
Opus 5.5 FS	medium	**893**	**359**	9.44

is 0.29 s. Most of the input is layout containers that show nothing and cannot be acted on. Removing them before encoding (without retraining) cuts the median input from 1,247 to 800 tokens on the held-out steps, costs 0.8 points alone (861 vs 869), and keeps routing intact (888 against 882 for Opus alone; 879 against 857 for Sonnet alone; 71.3% local). On the phone, the 8-bit model with pruned input takes a median 1.18 s (90th percentile 4.9 s), 2.9 times faster than unpruned, because attention cost grows faster than input length. Peak memory stays at 1.45 GB, dominated by ONNX Runtime's memory pool rather than by activations: disabling the pool lowers it to 0.97 GB but doubles latency. Sub-second latency on the phone will need its neural accelerator or a smaller encoder.

As a demonstration, with the model on a laptop driving the phone over USB, the small model completed a five-step Clock task (open the Timers tab, enter 5:00, start the timer) from step instructions, answering every step locally in 0.22–0.31 s.

7 Live Multi-Step Tasks

Offline evaluation starts every step from the recorded history, so errors cannot compound. On AndroidWorld (Rawles et al., 2025) (116 live tasks) a related cascade, in which a goal-only variant of the small model acted when confident and otherwise the T3A agent with Gemini-3.5-Flash acted, completed fewer tasks than T3A alone (41.9% vs 54.8%, $p = 0.008$, 93 tasks valid in both arms); a registered variant with app-name grounding and a no-effect guard also trailed (50.4% vs 58.3%). Run-to-run variation is large: across 43 identically configured episodes with greedy decoding, 65% followed a different action sequence and 9% changed outcome. A second registered design split the work the other way: a frontier planner wrote one to five step instructions from a compressed screen and the small model executed them, re-planning on low confidence or an unchanged screen. In an offline pilot on goal-only AndroidControl steps it scored 49.0% against 61.3% for the planner's model acting alone, with 5.6 times fewer frontier tokens. Taps were close (65.4% vs 69.1%); the loss came from the planner's instructions for app launches, waits and scrolls, which did not follow the conventions the small model had learned. Jev-Mobile (Zhang, 2026) reports a similar split on AndroidWorld (79% vs 84% at 73% lower cost) with a remote decision service as the executor rather than an on-device model. The offline results above should not be read as evidence for deployed multi-step agents.

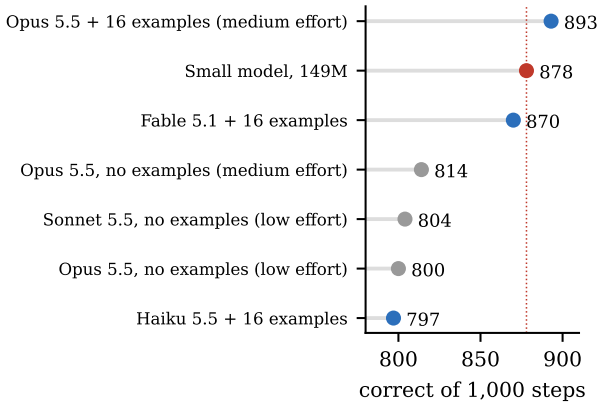


Figure 3: The sample used to set τ. Grey: frontier models without examples. Blue: with 16 retrieved examples. Red: the small model.

8 Discussion and Limitations

What the evidence supports. For single steps with a step instruction, a 149M model can take about 70% of a phone agent's steps while the routed system stays within 2 points of a frontier model that sees the same training data, at 15–30% of its cost. The result replicates, holds on unseen apps and, for Opus, on corrected labels, and survives 8-bit quantization. Most of the benefit is cost and data exposure: seven in ten screens never leave the device. The fallback must be strong; Haiku behind the small model adds nothing.

What it does not support. The small model alone is not better than a frontier model given the same examples, and its advantage on the steps it keeps comes from AndroidControl's labelling conventions (waiting, scroll direction), not from better grounding. Without step instructions the picture is mixed: in an earlier registered study on 1,000 fresh goal-only steps, a goal-only variant of the small model with Gemini-3.5-Flash as fallback scored 67.0% against 58.0% for Flash with eight fixed examples (+9.0, [+6.3, +11.8]), calling Flash on 38% of steps, but that baseline was weaker than the retrieved-example baseline used here, and the same design lost live (Sec. 7). The model is too slow on a phone CPU for interactive use.

Limitations. Inputs are text only, so unlabelled icons are hard. The small model's design was selected using the AndroidControl test split during development, so the held-out steps are fresh for the threshold and the frontier models but not for the design. Both the small model and the frontier prompts rely on 74,714 labelled in-domain steps. The frontier models were given text only; a screenshot could help them most on the deferred steps, many of which are taps on unlabelled icons, so a multimodal frontier baseline may be stronger than ours. The headline small model is a single training seed; three seeds of the same recipe on a 18,481-step training subset gave $80.8 \pm 0.7\%$. The frontier models are proprietary API versions that may change. Costs exclude on-device compute and the latency of a second call. Phone measurements cover one device and one runtime. This paper draws on a log of about 40 registered experiments and amendments, many with negative outcomes (Table 6); each result reported here was confirmed on data not used to design it, but we apply no correction across the program.

Broader Impact

A phone agent acts for a person and can expose what is on the screen. Keeping most steps local reduces what reaches a cloud provider but does not remove it: 29.3% of screens were still sent. Automated actions can cause harm (purchases, messages, deletions), and we did not evaluate safeguards. The live demonstration used an app that requires no account, and no personal data was recorded. AndroidControl is released under the Apache 2.0 license.

Table 6: Hypotheses not supported and ideas that did not help.

Hypothesis or idea	Outcome
Small model better than few-shot frontier models	Not supported (878 vs 893 for Opus, $p = 0.17$)
No accuracy drop on apps unseen in training	Not supported on a 72-app holdout (-3.6 pts)
Cascade to Haiku 4.5, then Sonnet (AndroidControl)	842, below the small model alone
Routing to Haiku 5.5 matches Opus	Not supported (-1.7 $[-3.5, +0.1]$)
Live cascade, distillation, self-training, completion guard	Below the frontier agent or no change
Retrieval memory for the small model; ModernBERT-large	$+0.0$ to $+0.3$ pts; 81.7% vs 88.5%

Reproducibility

We release the registration log with its commit history, the evaluation harness and prompts, the per-step outputs and billed token counts of every frontier request, the routing and re-scoring scripts, the trained model (safetensors and ONNX) with a one-call Python interface, the phone benchmark and the demonstration script. Tables 1–3 and the routing curves are recomputed by one script from the released files without new model calls, and a test suite asserts them; the curated-label, unseen-app and latency analyses also need the AndroidControl text file (built by a released notebook) or a phone.

Use of AI Assistance

The author designed and directed this work. Code, analyses and drafting were done with substantial help from Claude (Anthropic), and independent AI reviews were used to check the analysis and presentation. The author is responsible for the content.

References

Pranjal Aggarwal, Aman Madaan, et al. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems*, 2024.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

W. Chi et al. The impact of element ordering on LM agent performance. *arXiv preprint arXiv:2409.12089*, 2024.

C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970.

Filippos Christianos et al. Lightweight neural app control. In *International Conference on Learning Representations*, 2025. arXiv:2410.17883.

Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are we really making much progress? a worrying analysis of recent neural recommendation approaches. In *ACM Conference on Recommender Systems*, 2019.

Xiang Deng et al. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, 2023.

Dujian Ding et al. Hybrid LLM: Cost-efficient and quality-aware query routing. In *International Conference on Learning Representations*, 2024.

Gucongcong Fan, Chaoyue Niu, Chengfei Lyu, Fan Wu, and Guihai Chen. CORE: Reducing UI exposure in mobile agents via collaboration between cloud and local LLMs. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2510.15455.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.

Aryo Pradipta Gema et al. Inverse scaling in test-time compute. *Transactions on Machine Learning Research*, 2025.

G. Gonzalez-Pumariega et al. On the reliability of computer use agents. *arXiv preprint arXiv:2604.17849*, 2026.

Yangqin Jiang and Chao Huang. OpenPhone: Mobile agentic foundation models. *arXiv preprint arXiv:2510.22009*, 2025.

A. Khanal, Y. Tao, and J. Zhou. Beyond pass@1: A reliability science framework for long-horizon LLM agents. *arXiv preprint arXiv:2603.29231*, 2026.

H. F. Leung, X. Xi, and F. Zuo. AndroidControl-Curated: Revealing the true potential of GUI agents through benchmark purification. *arXiv preprint arXiv:2510.18488*, 2025.

Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on UI control agents. In *NeurIPS Datasets and Benchmarks Track*, 2024. arXiv:2406.03679.

Zachary C. Lipton and Jacob Steinhardt. Troubling trends in machine learning scholarship. *ACM Queue*, 17(1), 2019.

G. Liu et al. LearnAct: Few-shot mobile GUI agent with a unified demonstration benchmark. In *Findings of the Association for Computational Linguistics*, 2026. arXiv:2504.13805.

Y. Liu et al. InfiGUI-R1: Advancing multimodal GUI agents from reactive actors to deliberative reasoners. *arXiv preprint arXiv:2504.14239*, 2025.

Gábor Melis, Chris Dyer, and Phil Blunsom. On the state of the art of evaluation in neural language models. In *International Conference on Learning Representations*, 2018.

Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *International Conference on Machine Learning*, 2020.

Kevin Musgrave, Serge Belongie, and Ser-Nam Lim. A metric learning reality check. In *European Conference on Computer Vision*, 2020.

Isaac Ong et al. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

Yujia Qin et al. UI-TARS: Pioneering automated GUI interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.

Christopher Rawles et al. AndroidWorld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations*, 2025.

Melanie Sclar et al. Quantifying language models' sensitivity to spurious features in prompt design. In *International Conference on Learning Representations*, 2024.

Benjamin Warner et al. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhui Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, and Yuanchun Li. AutoDroid-V2: Boosting SLM-based GUI agents via code generation. *arXiv preprint arXiv:2412.18116*, 2024.

Zhiyong Wu et al. OS-ATLAS: A foundation action model for generalist GUI agents. *arXiv preprint arXiv:2410.23218*, 2024.

R. Xu et al. Retrieval-augmented GUI agents with generative guidelines. In *Proceedings of EMNLP*, 2025. arXiv:2509.24183.

Zhen Yang et al. Ferret-UI Lite: Lessons from building small on-device GUI agents. *arXiv preprint arXiv:2509.26539*, 2025.

Linghua Zhang. Jev-Mobile: Jev as an executor for mobile GUI agents. *arXiv preprint arXiv:2609.30186*, 2026.

Hanzhang Zhou et al. MAI-UI technical report: Real-world centric foundation GUI agents. *arXiv preprint arXiv:2512.22047*, 2025.

A Training Details

ModernBERT-base fine-tuned for 2 epochs on the 74,714 AndroidControl training steps with AdamW (learning rate 5×10^{-5}, weight decay 0.01), batch size 1 with 8-step gradient accumulation, 200 warm-up steps and linear decay, fp16 mixed precision, on one NVIDIA T4 GPU, seed 0. Inputs are truncated to 28 tokens per element and 3,072 tokens in total. Losses: cross-entropy over action types; for taps, a multi-positive cross-entropy over element markers in which every element whose centre lies inside the target counts as correct; for typed text and app names, cross-entropy over start and end positions (weight 0.5).

B Registered Experiments

Table 7: Registrations this paper relies on; all entries, with timestamps and amendments, are in the released log.

Registration	Question	Outcome
AC, AC-Opus	Small model vs zero-shot frontier models; three-tier cascade	Small model ahead; cascade 842
AC-Fair (+ effort control)	Small model vs few-shot Opus 5.5 and Fable 5.1	Not better; examples add 7.9 pts
AC-Router	Routing to few-shot Opus on held-out steps	Non-inferior; 70.7% local
AC-Router-Sonnet	Routing to few-shot Sonnet	Non-inferior to Sonnet and Opus
AC-Router-Rerun	Second run of Opus and Sonnet	Replicated
AC-Router-Haiku	Routing to few-shot Haiku	Non-inferior to Haiku only
W3–W12	A separate web system (not reported here)	See log
E14b–E14d	Goal-only routing offline (Gemini 3.5 Flash / Pro)	Routing +9.0 over few-shot Flash
E13–E24 (live)	Live cascades, distillation, self-training, guards	Below frontier agent or null
E1, E4, E5	Copying vs generating; seeds; generator scale	Copying ahead of 3B/7B generators by 2.4–6.8 pts; 3 seeds $80.8 \pm 0.7\%$
E2	Goal-only small model vs LiMAC (63.1%)	Not supported: 61.7% [60.5, 63.0]
E3, E6–E12	Model scale, features, selective prediction	Design choices; see log